

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Pertemuan 2

Model Data, Coroutines, dan Flow

Memodelkan laporan dengan aman, menunggu server tanpa layar macet, dan mengikuti status yang terus berubah

Muhammad Habib Algifari, S.Kom., M.T.I.

Tahun 2026



Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

Minggu lalu kita mencatat ini di papan

Saat membahas laporan kerusakan, kalian menyebut data yang wajib ada:

Data	Contoh isinya
Nomor laporan	7
Judul	Proyektor mati
Lokasi	GKU 1 ruang 201
Foto	boleh kosong
Status	Baru, Diproses, atau Selesai

Yang masih kita pegang dari P1

Proyek KMP yang jalan di Android atau desktop, dan pembagian kode di commonMain dan folder platform. Semua kode hari ini ditulis di commonMain.

MASALAH MINGGU INI

Status laporan berubah, aplikasi harus ikut tahu

Laporan proyektor masuk pukul 08.10, diterima petugas pukul 08.40, selesai pukul 10.15. Bagaimana aplikasi menunggu jawaban server tanpa layar macet, dan bagaimana ia tahu setiap kali status berubah?

Saat pulang nanti, kalian bisa

1

Memodelkan data

Menulis data class, sealed class, interface, dan tipe nullable untuk laporan.

2

Menunggu tanpa macet

Menulis suspend function, menjalankan dua proses bersamaan, dan menangani error.

3

Mengikuti perubahan

Membuat Flow status laporan dan mengolahnya dengan operator.

Capaian ini mendukung **CPMK0501** (menerapkan) dan **CPMK0502** (menjelaskan) konsep pemrograman untuk pengembangan perangkat lunak.

Bagian 1

Memodelkan laporan dengan Kotlin

Kotlin seperlunya: data class, sealed class, null safety, dan extension function untuk satu laporan.

data class untuk satu laporan

commonMain/model/Laporan.kt

```
data class Laporan(  
    val id: Int,  
    val judul: String,  
    val lokasi: String,  
    val fotoUrl: String?,  
    val status: StatusLaporan = StatusLaporan.Baru  
)
```

Satu kata **data** memberi kita **equals**, **toString**, dan **copy** tanpa menulisnya sendiri.

Semua properti memakai **val**. Laporan tidak diubah di tempat, tetapi disalin dengan nilai baru lewat **copy**. Kebiasaan ini akan sangat membantu di minggu 4.

sealed class untuk status yang terbatas

commonMain/model/StatusLaporan.kt

```
sealed class StatusLaporan {  
    data object Baru : StatusLaporan()  
    data class Diproses(  
        val petugas: String  
    ) : StatusLaporan()  
    data class Selesai(  
        val catatan: String  
    ) : StatusLaporan()  
}
```

Status laporan hanya ada tiga kemungkinan, dan sebagian membawa data sendiri: siapa petugasnya, apa catatan penyelesaiannya.

Karena daftarnya tertutup, **when** bisa memeriksa semua kemungkinan. Kalau suatu saat kita menambah status Ditolak, compiler langsung menunjuk setiap **when** yang belum menanganinya.

Foto boleh kosong, dan Kotlin memaksa kita jujur

commonMain

```
val foto: String? = laporan.fotoUrl  
  
val adaFoto = foto?.isNotBlank()  
val teks = foto ?: "Belum ada foto"  
val panjang = foto!!.length
```

Penulisan	Artinya
String?	Nilainya boleh null
?.	Jalankan hanya kalau tidak null
?:	Pakai nilai cadangan kalau null
!!	Saya yakin tidak null. Kalau salah, aplikasi crash

Menambah kemampuan tanpa mengubah class

commonMain/model/LaporanExt.kt

```
fun StatusLaporan.label(): String = when (this) {  
    StatusLaporan.Baru → "Menunggu petugas"  
    is StatusLaporan.Diproses → "Ditangani $petugas"  
    is StatusLaporan.Selesai → "Selesai: $catatan"  
}  
  
fun Laporan.ringkas(): String = "$id $judul, $lokasi"  
  
val belumDitangani = semuaLaporan.filter {  
    it.status is StatusLaporan.Baru  
}
```

`label()` dan `ringkas()` adalah extension function. Kita menempelkan fungsi ke tipe yang sudah ada, dan memanggilnya seolah fungsi bawaan.

Blok `filter { }` adalah lambda, yaitu fungsi tanpa nama yang dikirim sebagai argumen. Pola ini akan kita pakai terus di Flow dan Compose.

Tebak dulu sebelum dijalankan

Main.kt

```
val a = Laporan(7, "Proyektor mati", "GKU 1 R201", null)
val b = a.copy(status = StatusLaporan.Diproses("Pak Joko"))

println(a == b)
println(a.status.label())
println(b.status.label())
println(a.fotoUrl?.length ?: 0)
```

TEBAK 2 MENIT

Tulis empat baris keluaran di kertas sebelum kode ini dijalankan. Apakah **copy** mengubah laporan **a**?

Bagian 2

Menunggu server tanpa membuat layar macet

Coroutines: suspend function, launch dan async, dispatcher, dan structured concurrency.

Kenapa kita butuh coroutines

Mengambil data laporan dari server bisa makan satu sampai dua detik. Kalau proses itu berjalan di thread yang sama dengan tampilan, layar membeku, tombol tidak merespons, dan Android akhirnya menampilkan dialog aplikasi tidak merespons.

Coroutine membuat kita menulis kode yang menunggu secara berurutan, tanpa menahan thread yang sedang dipakai.

Analoginya

Pelayan rumah makan yang biasa menunggu di satu meja sampai masakan matang, baru pindah. Pelayan yang cekatan mencatat pesanan, melayani meja lain selagi menunggu, lalu kembali saat masakan siap. Thread adalah pelayannya, coroutine adalah cara kerjanya yang cekatan.

Menambahkan coroutines ke proyek

composeApp/build.gradle.kts

```
kotlin {  
    sourceSets {  
        commonMain.dependencies {  
            implementation(libs.kotlinx.coroutines.core)  
        }  
    }  
}
```

Istilah	Fungsinya
suspend	Fungsi yang boleh berhenti sejenak
launch	Mulai coroutine tanpa hasil
async	Mulai coroutine yang mengembalikan hasil
Dispatchers	Menentukan thread yang dipakai
CoroutineScope	Wadah yang mengatur umur coroutine

suspend function, fungsi yang bisa menunggu

commonMain/data/LaporanSumber.kt

```
suspend fun ambilLaporan(id: Int): Laporan {  
    delay(1000) // pura-pura menunggu server  
    return Laporan(id, "Proyektor mati", "GKU 1 R201", null)  
}  
  
suspend fun ambilRiwayat(id: Int): List<String> {  
    delay(800)  
    return listOf("Dikirim 08.10", "Diterima 08.40")  
}
```

`delay` menunda coroutine, bukan thread. Selama menunggu, thread bebas mengerjakan hal lain.

Aturannya satu: suspend function hanya bisa dipanggil dari coroutine atau dari suspend function lain.

launch untuk menyuruh, async untuk meminta hasil

Main.kt

```
fun main() = runBlocking {  
    launch { println("Notifikasi terkirim") }  
  
    val laporan = async { ambilLaporan(7) }  
    val riwayat = async { ambilRiwayat(7) }  
  
    println(laporan.await().judul)  
    println(riwayat.await())  
}
```

	launch	async
Hasil	Tidak ada	Deferred, diambil dengan await
Dipakai untuk	Tugas tanpa jawaban	Mengambil data bersamaan

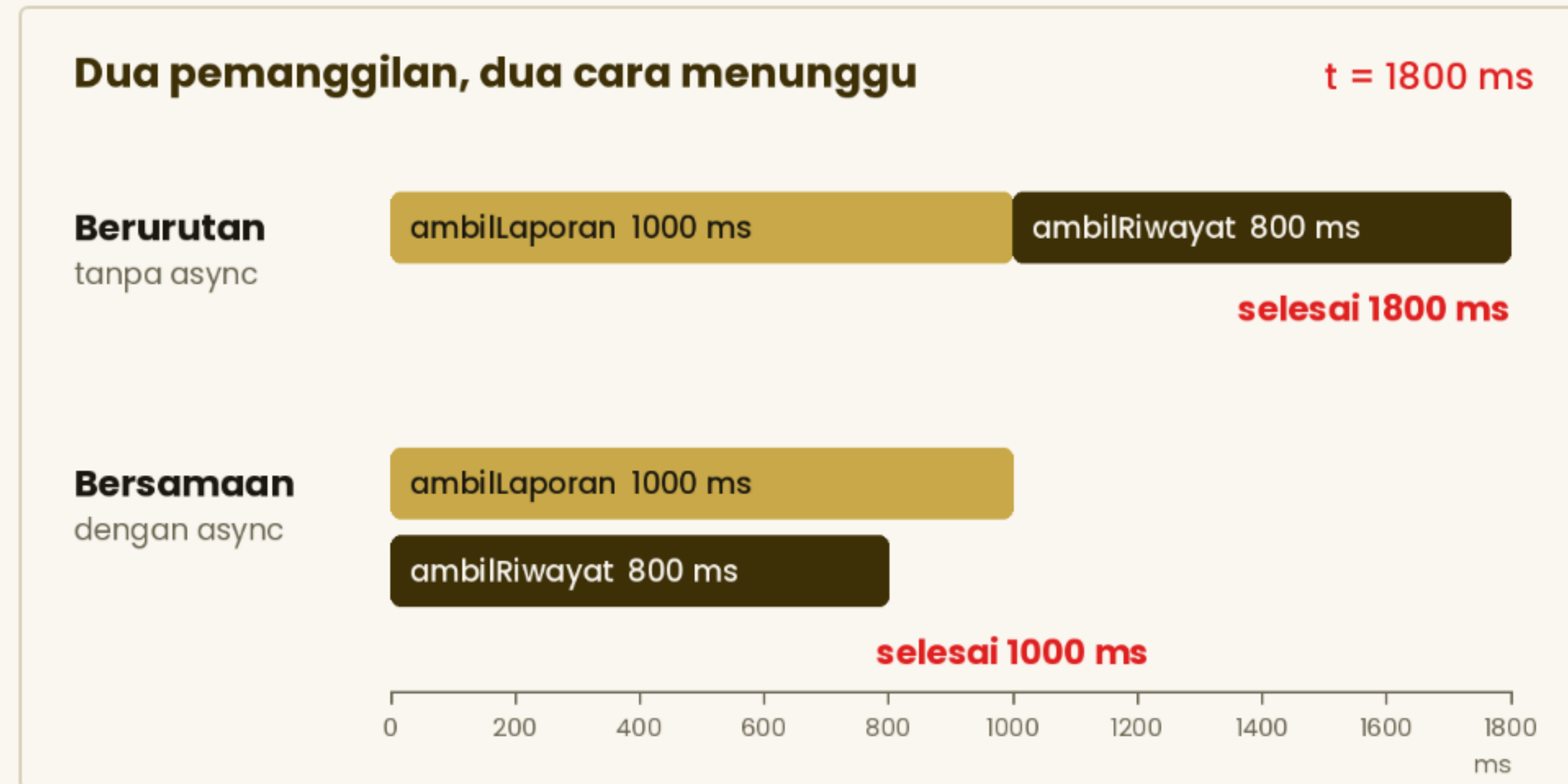
`runBlocking` hanya dipakai di fungsi main dan pengujian, tidak di aplikasi.

BANGUN BARENG

Berapa lama program tadi berjalan?

ambilLaporan menunggu 1000 ms dan ambilRiwayat menunggu 800 ms. Apakah totalnya 1800 ms, 1000 ms, atau 800 ms? Lalu baris mana yang tercetak paling dulu?

Lihat jalannya: 1800 ms menjadi 1000 ms



Animasi berputar saat presentasi. Di PDF yang tampil keadaan akhirnya.

COBA LANGSUNG



Scan atau klik, tekan Run, lalu ganti delay(800) menjadi delay(1500). Berapa hasilnya sekarang?

[Buka di Kotlin Playground](#)

Sumber: Kotlin Playground, JetBrains. Bacaan lanjut: [Coroutines basics, kotlinlang.org](#)

Dispatcher menentukan siapa yang bekerja

Dispatcher	Dipakai untuk	Contoh di LaporKampus
Main	Memperbarui tampilan	Menampilkan daftar laporan
IO	Jaringan, database, berkas	Mengambil laporan dari server
Default	Hitungan berat	Mengurutkan ribuan laporan

commonMain

```
suspend fun muatDetail(id: Int): Laporan =  
    withContext(Dispatchers.IO) {  
        ambilLaporan(id)  
    }
```

Structured concurrency, setiap coroutine punya induk

1 **Punya induk**
Setiap coroutine berjalan di dalam satu scope.

2 **Induk menunggu**
Scope belum selesai sebelum semua anaknya selesai.

3 **Batal bersama**
Kalau induk dibatalkan, semua anaknya ikut berhenti.

4 **Gagal diteruskan**
Kalau satu anak gagal, kegagalannya diteruskan ke induk.

Di LaporKampus: saat pengguna menutup layar detail laporan, semua pengambilan data untuk layar itu ikut dibatalkan. Tidak ada proses yang terus berjalan diam-diam dan memboroskan baterai.

Bagian 3

Mengikuti status yang terus berubah

Flow: aliran beberapa nilai yang datang satu per satu, lengkap dengan operatornya.

Satu nilai atau banyak nilai

suspend function

Mengembalikan satu nilai, sekali, lalu selesai.

```
suspend fun ambilLaporan(id: Int): Laporan
```

Cocok untuk membuka detail satu laporan.

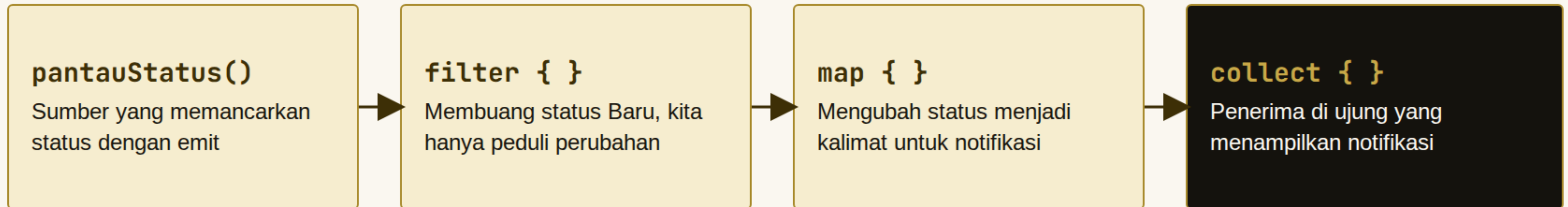
Flow

Memancarkan beberapa nilai berurutan dari waktu ke waktu.

```
fun pantauStatus(id: Int): Flow<StatusLaporan>
```

Cocok untuk status laporan yang berubah dari Baru, Diproses, sampai Selesai.

Jalur yang dilalui setiap status



Nilai mengalir dari kiri ke kanan. Tanpa collect di ujung, tidak ada yang mengalir sama sekali.

Membuat Flow status laporan

commonMain/data/LaporanSumber.kt

```
fun pantauStatus(id: Int): Flow<StatusLaporan> = flow {  
    emit(StatusLaporan.Baru)  
    delay(1500)  
    emit(StatusLaporan.Diproses("Pak Joko"))  
    delay(2000)  
    emit(StatusLaporan.Selesai("Lampu proyektor diganti"))  
}
```

Di dalam blok `flow { }` kita memanggil `emit` setiap kali ada nilai baru.

Cara lain yang lebih singkat: `flowOf(a, b, c)` dan `daftar.asFlow()`.

collect membuka keran

Main.kt

```
fun main() = runBlocking {  
    val aliran = pantauStatus(7)  
    println("Sebelum collect")  
    aliran.collect { status →  
        println(status.label())  
    }  
    println("Sesudah collect")  
}
```

Keluaran

Sebelum collect
Menunggu petugas
Ditangani Pak Joko
Selesai: Lampu proyektor diganti
Sesudah collect

Baris Sesudah collect baru tercetak setelah Flow selesai, karena **collect** ikut menunggu.

Operator untuk mengolah aliran

Operator	Gunanya
map	Mengubah setiap nilai
filter	Meloloskan nilai yang memenuhi syarat
take	Mengambil beberapa nilai pertama
onEach	Efek samping, misalnya mencatat log
catch	Menangani error di aliran

commonMain

```
pantauStatus(7)
    .filter { it !is StatusLaporan.Baru }
    .map { it.label() }
    .onEach { println("Log: $it") }
    .collect { tampilkanNotifikasi(it) }
```

Satu kontrak untuk semua sumber data

commonMain/data/Sumber.kt

```
interface Sumber<T> {  
    suspend fun ambil(id: Int): T  
    fun pantau(id: Int): Flow<T>  
}  
  
class SumberLaporanPalsu : Sumber<Laporan> {  
    override suspend fun ambil(id: Int): Laporan {  
        delay(1000)  
        return contohLaporan(id)  
    }  
    override fun pantau(id: Int) = flow {  
        emit(contohLaporan(id))  
    }  
}
```

interface adalah janji: siapa pun yang mengaku Sumber wajib punya **ambil** dan **pantau**.

<T> membuat janji yang sama berlaku untuk Laporan, Ruang, Buku, atau entitas tema kalian. Minggu 6 kita menulis SumberLaporanServer tanpa mengubah pemanggilnya.

Kalau server gagal: try, catch, dan pembatalan

commonMain/data/MuatAman.kt

```
suspend fun muatAman(id: Int): String =  
    try {  
        require(id > 0) { "id harus positif" }  
        sumber.ambil(id).ringkas()  
    } catch (e: CancellationException) {  
        throw e // pembatalan jangan ditelan  
    } catch (e: Exception) {  
        "Gagal memuat laporan #$id"  
    }  
}
```

`try` di Kotlin menghasilkan nilai, jadi bisa langsung dipakai sebagai isi fungsi.

Di dalam coroutine, `CancellationException` harus dilempar ulang. Kalau ditelan, coroutine yang seharusnya batal akan terus berjalan dan structured concurrency rusak.

Bagian 4

Coba sendiri, lalu bawa ke proyekmu

Latihan Kotlin dan coroutines dengan observasi praktik, model data tema, dan checkpoint proyek.

Yang saya amati selama latihan hari ini

Indikator RPS: **Kode dengan Coroutines dan Flow berfungsi.** Diamati dari latihan 1 dan 2, coroutines dan Flow.

Kriteria	Sangat baik 85 sampai 100	Baik 70 sampai 84	Cukup 55 sampai 69	Kurang di bawah 55
Coroutines dan Flow berfungsi CPMK0501, 2%	Latihan 1 selesai sekitar 1000 ms dan latihan 2 menyaring serta memetakan dengan benar	Keduanya jalan, satu hasil kurang tepat	Hanya satu latihan yang jalan	Belum ada yang jalan
Menjelaskan coroutines dan Flow CPMK0502, 2%	Menjelaskan kenapa async lebih cepat dan kapan Flow mulai mengalir, dengan kode sendiri	Penjelasan benar, ada istilah yang keliru	Hanya membacakan kode	Tidak bisa menjelaskan

Individu, bobot 4% nilai akhir. Bukti: push ke repository latihan pribadimu, lalu kirim tautan commit di LMS sebelum kelas selesai.

Tiga latihan di repo mata kuliah

Coroutines, latihan 1

handson1-latihan

Ambil profil dan postingan bersamaan. Targetnya sekitar 1000 ms, bukan 1800 ms.

Flow, latihan 2

handson2-latihan

Saring suhu di atas 30 derajat, ubah menjadi pesan peringatan, lalu tampilkan.

Kotlin Dasar P2, latihan 3

handson3-latihan

sealed class NetworkResult dengan Loading, Success, dan Error, ditangani dengan when.

Dua latihan pertama di folder **P2 Advanced Kotlin Coroutines Flow Hands-on**. Latihan ketiga di folder **Kotlin Dasar, P2 Object-Oriented Programming Hands-on**.

Kalau error

Pesan error	Penyebab	Yang dicoba
Suspend function can only be called from coroutine	Memanggil suspend function dari fungsi biasa	Panggil dari dalam launch, runBlocking, atau scope lain
Flow invariant is violated	Memanggil launch di dalam blok flow	Pakai channelFlow untuk pancaran dari beberapa coroutine
JobCancellationException	Scope induk dibatalkan sebelum anak selesai	Periksa umur scope, rapikan dengan try dan finally

Tulis model data tema kelompokmu

1 **Entitas utama**
Satu data class untuk entitas utama tema kalian, lengkap dengan tipe datanya.

2 **Entitas pendukung**
Data class kedua di tema kalian, misalnya Ruang atau MataKuliah.

3 **Satu sealed class**
Status tiga nilai sesuai Bank Proyek, misalnya Diajukan, Disetujui, Ditolak.

4 **Tandai yang boleh kosong**
Tipe nullable hanya untuk kolom yang memang boleh kosong.

CONTOH DARI LAPORKAMPUS

Laporan adalah entitas utamanya, StatusLaporan adalah sealed class-nya, dan fotoUrl satu-satunya kolom nullable.

Checkpoint proyek minggu 2

1 Model data tema

Dua data class dan satu sealed class status di commonMain/model.

2 Sumber data palsu

Implementasi Sumber untuk entitas utama, dengan delay dan Flow perubahan status.

3 Bukti berjalan

Fungsi main atau test kecil yang mencetak perubahan status ke konsol.

Dikumpulkan

Push ke repository kelompok. README memuat contoh keluaran konsol dan siapa mengerjakan bagian mana.

Dicek di awal pertemuan berikutnya: tuntas, sebagian, atau belum.

TUTUP

Tiket keluar



1 **Model data**
Kenapa status laporan lebih aman ditulis sebagai sealed class daripada String?

2 **Coroutines**
Kenapa dua async yang menunggu 1000 ms dan 800 ms selesai dalam sekitar 1000 ms?

3 **Flow**
Apa yang terjadi pada sebuah Flow kalau tidak ada yang memanggil collect?

Jawab singkat di kertas atau formulir LMS sebelum keluar kelas.

TUTUP

Minggu depan: dasar Compose Multiplatform



MASALAH MINGGU 3

Data laporan sudah ada. Sekarang bagaimana menampilkannya sebagai daftar kartu yang rapi, dengan judul, lokasi, dan label status, di layar Android dan desktop sekaligus?

Persiapan

Bawa model data dan sumber palsu kelompok kalian. Minggu depan masih ada Kotlin seperlunya: lambda dan pengolahan koleksi.

Terima kasih

Sampai jumpa di pertemuan 3

Ada pertanyaan? Sampaikan sekarang, lewat LMS, atau email ke muhammad.algifari@if.itera.ac.id